

CONCEPTOS DE BASES DE DATOS

CLASE 2



- Operaciones usuales a resolver
 - **Modificar** el contenido actual de un archivo
 - **Agregar** nuevos elementos a un archivo
 - **Actualizar** un archivo maestro con uno o varios archivos detalles
 - **Corte de control**
 - **Merge** de archivos

- Algunas consideraciones generales
 - Las declaraciones de tipos y relaciones de los archivos con el SO (**ASSIGN**) se encuentran en el programa principal.
 - No intentar hacer lecturas en un archivo sin antes chequear que hay registros disponibles (**EOF**).
 - Cerrar archivos (**CLOSE**) al finalizar su uso en los algoritmos.
 - Para incrementar la eficiencia de los algoritmos, se debe minimizar los accesos a los archivos **maestro**:
 - Si es posible, no recorrerlos más de una vez.
 - Si es posible, no recorrerlos de forma completa.

- Ej: actualización masiva de salarios

Una empresa cuenta con un **sistema de recursos humanos** en donde almacena la información de sus empleados en un archivo de datos llamado “**empleados.dat**”. Cada registro contiene información como:

- Apellido y nombre
- Domicilio
- Salario
- Otros datos administrativos

La empresa decide **aplicar un aumento salarial del 20% a todos los empleados.**

Características del proceso:

- El archivo **ya contiene información cargada**.
- Los registros **no están ordenados** por ningún criterio.
- Se debe modificar **cada registro existente**.
- No se agregan ni eliminan registros.

El archivo debe ser **recorrido de forma completa** siguiendo un **procesamiento secuencial**.

Archivos

Se debe procesar registro por registro, del primero al último.

Para la modificación de un registro:

- Se lee el registro.
- Se realiza la modificación de datos.
- Se vuelve a la posición del registro.
- Se escribe el registro actualizado.

Modificación

EMPLEADOS

Gomez, Juan Calle 15 nro 432 \$120.000 ...
Alvarez, Diego Calle 70 nro 258 \$143.000 ...
Zapata, Javier Av. 13 nro 1679 \$170.000 ...
...
...
...
...
EOF

Archivos

{declaración de tipos de datos en el programa principal, que utiliza el proceso "actualizar"}

```
Program mainProgram;  
type  
    registro = record  
        nombre: string[20];  
        direccion: string[20];  
        salario: real;  
        (...)  
    end;  
    empleados = file of registro;  
var  
    archivoEmp: empleados;  
begin  
    assign(archivoEmp, "empleados.dat");  
    actualizar (archivoEmp, 1.2);  
    (...)
```

Modificación

{se recibe el archivo como parámetro por referencia, se recibe un valor porcentual de actualización (1.2 → 20%) }

```
Procedure actualizar (var fileEmp:empleados, valor:real);  
var  
    regEmp: registro;  
begin  
    reset(fileEmp);  
    while not eof(fileEmp) do  
        begin  
            read(fileEmp, regEmp);  
            regEmp.salario := regEmp.salario * valor;  
            seek(fileEmp, filepos(Emp) -1 );  
            write(fileEmp, regEmp);  
        end;  
    close(fileEmp);  
end;
```

- Ej: incorporación de nuevos empleados

Continuando con el sistema de recursos humanos, ahora la empresa decide **contratar nuevos empleados**. La información de los nuevos empleados debe quedar guardada en el archivo **“empleados.dat”**.

Características del proceso:

- El archivo **ya contiene información cargada**.
- Los registros existentes deben **conservarse**.
- Los nuevos registros deben **agregarse al final del archivo**.
- No es necesario mantener **ningún orden específico**.

Archivos

Agregar datos

Se debe permitir al usuario **ingresar nuevos empleados desde teclado**. El proceso finaliza cuando el usuario indica que no desea ingresar más empleados.

Al iniciar se debe posicionar el puntero sobre la marca de **EOF**.

Luego, para la inserción de cada nuevo registro:

- Se leen los datos completos desde teclado.
- Se realiza la escritura.

Al finalizar la incorporación de todos los nuevos empleados, se **cierra el archivo**.

EMPLEADOS

Gomez, Juan Calle 15 nro 432 \$120.000 ...
Alvarez, Diego Calle 70 nro 258 \$143.000 ...
Zapata, Javier Av. 13 nro 1679 \$170.000 ...
...
...
...
...
EOF

Archivos

```
Procedure agregar(var fileEmp: empleados)
var
    regEmp: registro;
begin
    reset(fileEmp);
    seek(fileEmp, filesize(fileEmp));
    leer_datos(regEmp);
    while (regEmp.nombre <> ' ') do
        begin
            write(fileEmp, regEmp);
            leer_datos(regEmp);
        end;
    close(fileEmp);
end;
```

Agregar datos

- El procedimiento **leer_datos()** realiza la lectura desde teclado de un registro de empleado en forma completa, campo por campo.
- Cuando **leer_datos()** devuelve un nombre de empleado **vacío**, significa que el usuario no desea agregar más empleados.

Archivos

Procedure leer_datos(var regEmp: registro)

var

regEmp: registro;

begin

write("Ingrese el nombre del
empleado:");

read(regEmp.nombre);

if (regEmp.nombre <> '')

begin

write("Ingrese su dirección:");

read(regEmp.direccion);

(...)

end;

end;

Agregar datos

- El procedimiento **leer_datos()** realiza la lectura desde teclado de un registro de empleado en forma completa, campo por campo.
- Cuando **leer_datos()** devuelve un nombre de empleado **vacío**, significa que el usuario no desea agregar más empleados.

Archivos

Actualización maestro-detalle

El problema de la **actualización maestro-detalle** involucra utilizar al mismo tiempo varios archivos de datos de distinto tipo.

Se denomina **maestro** al archivo que **resume información sobre un dominio específico**. Generalmente estos archivos almacenan datos de:

- Información sobre algún **aspecto importante de las actividades** de una organización: **CLIENTES, PRODUCTOS, VENDEDORES, PROVEEDORES**, etc.
- Información que refleja la **historia de eventos** de la organización: **PEDIDOS, VENTAS**, etc.

Archivos

Actualización maestro-detalle

Se denomina **detalle** al archivo que contiene **nueva información** generada por las diferentes aplicaciones.

- Cada aplicación crea y maneja sus **propios archivos detalle** y los utiliza para la operatoria diaria.
- Al finalizar la jornada, se usan para actualizar la información de los archivos maestro.

En general, las actualizaciones involucran:

- Un único archivo maestro.
- Varios archivos detalle.

Archivos

Un maestro - Un detalle

- Ej: actualización de horas trabajadas

Una empresa almacena la información de sus empleados en un archivo maestro denominado “**empleados.dat**”, que contiene todos los datos de cada empleado, entre ellos:

- Nombre
- Dirección
- Cantidad total de horas trabajadas
- Otros datos administrativos

Archivos

Un maestro - Un detalle

Durante la jornada laboral, un **sistema de control de acceso** registra las **horas trabajadas durante el día**.

Al finalizar la jornada, esta información se guarda en un archivo de detalle denominado “**detalleCHT.dat**”, en donde cada registro contiene:

- Nombre del empleado
- Cantidad de horas trabajadas durante el día

Se debe **actualizar la cantidad total de horas trabajadas** por cada empleado en el archivo maestro.

Archivos

Un maestro - Un detalle

Si los archivos involucrados estuvieran desordenados, este proceso involucraría recorrer el archivo maestro en más de una ocasión.

Precondiciones

- Ambos archivos (maestro y detalle) están **ordenados** por el mismo criterio, en este caso el **nombre del empleado**.
- En el archivo detalle solo aparecen empleados que **existen** en el archivo maestro.
- Cada empleado del archivo maestro a lo sumo puede aparecer **una vez** en el archivo detalle.

Archivos

Un maestro - Un detalle

- Se debe recorrer el archivo **maestro** buscando el primero de los registros del archivo **detalle**
- Una vez encontrado, se realiza la actualización del registro:
 - Se modifican los datos.
 - Se vuelve a la posición del registro.
 - Se escribe el registro actualizado.
- Se continua con el siguiente registro del archivo **detalle**.
- Cuando no hay más registros en el archivo **detalle**, se finaliza.

MAESTRO

Alvarez, Diego Calle 70 nro 258 70 ...
Gomez, Juan Calle 15 nro 432 60 ...
Rodriguez, Carlos Calle 72 nro 320 150 ...
Tapia, Ricardo Av. 7 nro 179 87 ...
Zapata, Javier Av. 13 nro 1679 170 ...
...
EOF

DETALLE

Gomez, Juan 10
Tapia, Ricardo 15
Zapata, Javier 20
EOF

Archivos

Un maestro - Un detalle

Program actualizar;

type

 empleado = **record**

 nombre: string[30];

 direccion: string[30];

 cht: integer; *{cantidad de horas trabajadas}*

end;

 empDia = **record**

 nombre: string[30];

 cht: integer;

end;

 maestro = file of empleado; *{archivo que contiene la info completa}*

 detalle = file of empDia; *{archivo que contiene la info diaria}*

var

 mae1: maestro;

 det1: detalle;

 regm: empleado;

 regd: empDia;

Archivos

Un maestro - Un detalle

begin

assign (mae1, empleados.dat'); *{se asocian y se abren los archivos}*

assign (det1, 'detalleCHT.dat');

reset (mae1);

reset (det1);

while (not eof(det1)) do *{mientras haya elementos en el detalle}*

begin

read(mae1, regm); *{siempre consultar EOF antes de leer, en este caso sabemos que no está vacío}*

read(det1,regd);

while (regm.nombre<> regd.nombre) do *{se busca el empleado en el maestro}*

read (mae1,regm);

regm.cht := regm.cht + regd.cht; *{al encontrarlo se actualiza}*

seek (mae1, filepos(mae1)-1);

write(mae1,regm);

end;

close(det1); *{se cierran los archivos}*

close(mae1);

end.

Archivos

Un maestro - Un detalle

- Ej: actualización de stock de productos

Un supermercado almacena la **información de sus productos** en un archivo maestro llamado “**productos.dat**”.

Este archivo contiene todos los datos de cada producto, entre ellos:

- Código de producto
- Descripción
- Cantidad
- Precio unitario
- Otros datos administrativos

Archivos

Un maestro - Un detalle

Durante el día se registran las ventas realizadas en las distintas cajas del supermercado. Cada venta genera un registro con:

- Código de producto
- Cantidad vendida

Al finalizar la jornada, todas las ventas se almacenan en un archivo llamado “**ventas.dat**”, que constituye el archivo detalle.

Se debe actualizar **el stock de cada producto** en el archivo maestro.

Archivos

Un maestro - Un detalle

Si los archivos involucrados estuvieran desordenados, este proceso involucraría recorrer el archivo maestro en más de una ocasión

Precondiciones

- El archivo maestro **no está vacío**.
- Ambos archivos están **ordenados** por código de producto.
- En el archivo detalle solo aparecen productos que **existen** en el archivo maestro.
- Cada producto puede ser, a lo largo del día, vendido más de una vez → en el archivo detalle pueden existir **varios registros correspondientes al mismo producto**.

Archivos

Un maestro - Un detalle

- Se procesan todos los registros del archivo **detalle** con igual código.
- Se encuentra el registro en el archivo **maestro** y se realiza la actualización:
 - Se modifican los datos.
 - Se vuelve a la posición del registro.
 - Se escribe el registro actualizado.
- Se continua con el siguiente registro del archivo **detalle**.
- Cuando no hay más registros en el archivo **detalle**, se finaliza.

MAESTRO

0012 Yogurt 80 ...
0045 Chocolate 100 ...
0214 Galletitas 90 ...
0501 Jabon 84 ...
0602 Toalla 30 ...
...
EOF

DETALLE

0012 12
0214 20
0214 10
0501 5
EOF

Archivos

```
program actualizar;
const
    valoralto='9999';
type
    str4 = string[4];
    prod = record
        cod: str4;
        descripcion: string[30];
        cant: integer;
        pu: real;
        (...)
    end;
    v_prod = record
        cod: str4;
        cant_vendida: integer;
    end;
maestro = file of prod;
detalle = file of v_prod;
```

Un maestro - Un detalle

```
var
    mae1: maestro;
    det1: detalle;

    regm: prod;
    regd: v_prod;

    total: integer;
    aux: str4;
```

Archivos

Un maestro - Un detalle

```
procedure leer (var archivo:detalle; var dato:v_prod);
```

```
begin
```

```
    if (not eof(archivo))
```

```
        then read (archivo,dato)
```

```
        else dato.cod:= valoralto;
```

```
end;
```

```
begin  {programa principal}
```

```
    assign (mae1, 'productos.dat'); {se asocian y se abren los archivos}
```

```
    assign (det1, 'ventas.dat');
```

```
    reset (mae1);
```

```
    reset (det1);
```

```
    read(mae1,regm); {siempre se debe preguntar por EOF antes de leer → en  
este caso el enunciado del ejemplo dice que no está vacío}
```

```
    leer(det1,regd);
```

Archivos

Un maestro - Un detalle

```
while (regd.cod <> valoralto) do {se procesan todos los registros del archivo det1}
begin
    aux := regd.cod;
    total := 0;
    while (aux = regd.cod ) do {se procesan códigos iguales del detalle}
    begin
        total := total + regd.cant_vendida;
        leer(det1,regd);
    end;
    while (regm.cod <> aux) do {se busca el registro detalle en el maestro}
        read (mae1,regm);
        regm.cant := regm.cant - total; {se modifica el stock del producto}
        seek (mae1, filepos(mae1)-1); {se reubica el puntero y actualiza}
        write(mae1,regm);
    end;
    close(det1); {se cierra los archivos}
    close(mae1);
```

Archivos

Un maestro - Un detalle

{se visualiza en pantalla el stock de cada producto}

```
reset(mae1);
```

```
while (not eof(mae1)) do
```

```
begin
```

```
    read (mae1,regm);
```

```
    writeln(regm.cod, regm.cant);
```

```
end;
```

```
close(mae1);
```

```
end.
```

Archivos

Un maestro - N detalles

- Ej: actualización de stock de productos <N detalles>

Continuando con el sistema del supermercado que almacena la **información de sus productos** en un archivo maestro llamado “**productos.dat**”, con la siguiente estructura:

- Código de producto
- Descripción
- Cantidad
- Precio unitario
- Otros datos administrativos

Archivos

Un maestro - N detalles

Durante el día se registran las ventas realizadas en las distintas cajas del supermercado. Cada venta genera un registro con:

- Código de producto
- Cantidad vendida

Pero ahora, al finalizar la jornada cada caja genera su propio archivo con las ventas realizadas.

Si se cuenta con 3 cajas ($N=3$), entonces se dispone de 3 archivos de detalle: “**ventas1.dat**”, “**ventas2.dat**” y “**ventas3.dat**”.

Se debe actualizar **el stock de cada producto** en el archivo maestro.

Archivos

Un maestro - N detalles

Si los archivos involucrados estuvieran desordenados, este proceso involucraría recorrer el archivo maestro en más de una ocasión

Precondiciones

- El archivo maestro **no está vacío**.
- Todos los archivos están **ordenados** por código de producto.
- En el archivo detalle solo aparecen productos que **existen** en el archivo maestro.
- Cada producto puede ser, a lo largo del día, vendido más de una vez → en los archivos detalle pueden existir **varios registros correspondientes al mismo producto**.

Archivos

Un maestro - N detalles

El algoritmo debe **comparar simultáneamente los registros actuales de cada archivo y seleccionar el menor** según el criterio de ordenamiento.

Este proceso se repite hasta **procesar completamente todos los archivos**.

A esta técnica se la conoce como **búsqueda del mínimo**.

Archivos

Un maestro - N detalles

- Se procesan todos los registros de los archivos **detalle** con igual código.
- Se encuentra el registro en el archivo **maestro** y se realiza la actualización:
 - Se modifican los datos.
 - Se vuelve a la posición del registro.
 - Se escribe el registro actualizado.
- Se continua con el siguiente registro de los archivos **detalle**.
- Cuando no hay más registros en los archivos **detalle**, se finaliza.

MAESTRO

0012 Yogurt 80 ...
0045 Chocolate 100 ...
0214 Galletitas 90 ...
0501 Jabon 84 ...
0602 Toalla 30 ...
...
EOF

DETALLE 1

0012 12
0214 20
EOF

DETALLE 2

0214 10
0501 5
EOF

DETALLE 3

0045 3
0214 5
EOF

Archivos

```
program actualizarN;  
const  
    valoralto='9999';  
type  
    str4 = string[4];  
    prod = record  
        cod: str4;  
        descripcion: string[30];  
        cant: integer;  
        pu: real;  
        (...)  
    end;  
    v_prod = record  
        cod: str4;  
        cant_vendida: integer;  
    end;  
    maestro = file of prod;  
    detalle = file of v_prod;
```

Un maestro - N detalles

```
var  
    mae1: maestro;  
    det1, det2, det3: detalle;  
  
    regm: prod;  
    min, reg1, reg2, reg3: v_prod;  
  
    aux: str4;
```

Archivos

Un maestro - N detalles

```
procedure leer (var archivo:detalle; var dato:v_prod);  
begin  
    if (not eof(archivo))  
    then  
        read (archivo,dato)  
    else  
        dato.cod:= valoralto;  
end;
```

Archivos

Un maestro - N detalles

```
procedure minimo (var r1, r2, r3:v_prod; var min:v_prod
                  var det1, det2, det3: detalle);
begin
  if (r1.cod <= r2.cod) and (r1.cod <= r3.cod)
  then begin
    min := r1;
    leer(det1,r1)
  end
  else if (r2.cod <= r3.cod)
  then begin
    min := r2;
    leer(det2,r2)
  end
  else begin
    min := r3;
    leer(det3,r3)
  end;
end;
```

Archivos

Un maestro - N detalles

```
begin {programa principal}
    assign (mae1, productos.dat'); {se asocian y se abren los archivos}
    assign (det1, 'ventas1.dat');
    assign (det2, 'ventas2.dat');
    assign (det3, 'ventas3.dat');
    reset (mae1);
    reset (det1);
    reset (det2);
    reset (det3);

    read(mae1, regm); {siempre se debe preguntar por EOF antes de leer → en este caso
    el enunciado del ejemplo dice que no está vacío}
    leer(det1, regd1);
    leer(det2, regd2);
    leer(det3, regd3);

    minimo(regd1, regd2, regd3, min, det1, det2, det3); {se obtiene el minimo}
```

Archivos

Un maestro - N detalles

```
while (min.cod <> valoralto) do
begin
  while (regm.cod <> min.cod) do {busca el mínimo en el maestro}
    read(mae1, regm);
  aux := min.cod;
  while (aux = min.cod ) do {procesa el mínimo}
  begin
    regm.cant := regm.cant - min.cantvendida;
    minimo(regd1, regd2, regd3, min, det1, det2, det3);
  end;
  seek (mae1, filepos(mae1)-1); {se reubica el puntero y actualiza}
  write(mae1, regm);
end;
close(mae1); {se cierran los archivos}
close(det1); close(det2); close(det3);
```

Archivos

Un maestro - N detalles

{se visualiza en pantalla el stock de cada producto}

```
reset (mae1);
```

```
while (not eof(mae1)) do
```

```
begin;
```

```
    read (mae1, regm);
```

```
    writeln (regm.cod, regm.cant);
```

```
end;
```

```
close(mae1);
```

```
end.
```

- Algorítmica clásica de corte de control

Este tipo de algoritmos permite analizar la información almacenada en archivos y **generar reportes** que incluyan valores promedios, subtotales, totales, etc.

Precondiciones:

- El archivo se encuentra **ordenado por uno o más criterios**.
- Los criterios de orden utilizados **deben ser los mismos criterios** por los que se quiere agrupar la información en el reporte.

Archivos

Corte de control

El algoritmo consiste en ir recorriendo la información, de forma tal que cada vez que se produce un **cambio en alguno de los campos correspondiente a un criterio de ordenamiento**, se finaliza el cálculo relacionado y se comienza el siguiente.

Para realizar esto, se utilizan **bucles anidados** que van **incrementando la cantidad de condiciones a verificar**. Cada bucle controla un nivel de agrupamiento del archivo.

El resultado es un reporte que respeta un **formato predeterminado**.

Archivos

Corte de control

- Ej: reporte de ventas

Una consultora analiza las **ventas realizadas por distintas empresas en sus sucursales**. Se dispone de un archivo llamado “**ventas.dat**” con la siguiente información:

- Empresa
- Sucursal
- Producto
- Cantidad vendida

El archivo se encuentra ordenado por **Empresa**, luego por **Sucursal** y luego por **Producto**.

Archivos

Corte de control

Se desea generar un reporte que muestre:

- Total vendido por producto
- Total vendido por sucursal
- Total vendido por empresa
- Total general

Hay tres niveles de corte a implementar:

- Empresa
- Sucursal
- Producto

Archivos

Corte de control

- Ejemplo de los datos que contiene el archivo

Empresa	Sucursal	Producto	Cantidad
MegaPC	Centro	Mouse	3
MegaPC	Centro	Notebook	2
MegaPC	Centro	Notebook	10
MegaPC	Norte	Notebook	1
MegaPC	Norte	Teclado	5
TechStore	Centro	Mouse	4
TechStore	Centro	Notebook	6

Archivos

Empresa: TechStore

Sucursal: Centro

Producto: Notebook

Total producto: 5

Producto: Mouse

Total producto: 10

Total sucursal: 15

Sucursal: Norte

Producto: Notebook

Total producto: 1

Producto: Teclado

Total producto: 5

Total sucursal: 6

Total empresa: 21

Corte de control

Empresa: MegaPC

Sucursal: Centro

Producto: Notebook

Total producto: 4

Producto: Mouse

Total producto: 6

Total sucursal: 10

Total empresa: 10

TOTAL GENERAL: 31

Archivos

Corte de control

```
program corteDeControl;  
const  
    valorAlto='zzzz';  
  
type  
    regVentas = record  
        empresa: string;  
        sucursal: string;  
        producto: string;  
        cantidad: integer;  
    end;  
    fileVentas = file of regVentas;
```

Archivos

Corte de control

var

ventas: fileVentas;

reg: regVentas;

empActual, sucActual, prodActual : string;

totalProd, totalSuc, totalEmp, totalGeneral : integer;

procedure leer (var archivo: fileVentas; var dato: regVentas);

begin

if (not eof(archivo))

then read (archivo, dato);

else dato.empresa := valorAlto;

end;

Archivos

Corte de control

begin

assign (ventas, 'ventas.dat');

reset(ventas);

leer(ventas, reg);

totalGeneral := 0;

while (reg.empresa <> valorAlto) do

begin

empActual := reg.empresa;

totalEmp := 0;

writeln('Empresa: ', empActual);

while (reg.empresa = empActual) do

begin

sucActual := reg.sucursal;

totalSuc := 0;

writeln(' Sucursal: ', sucActual);

// bucle 1 = fin de archivo

// bucle 2 = empresa

Archivos

Corte de control

```
writeln(' Sucursal: ', sucActual);
while (reg.empresa = empActual) and (reg.sucursal = sucActual) do // bucle 3 = sucursal
begin
    prodActual := reg.producto;
    totalProd := 0;
    writeln(' Producto: ', prodActual);
    while (reg.empresa = empActual) and (reg.sucursal = sucActual) and
    (reg.producto = prodActual) do // bucle 4 = producto
    begin
        totalProd := totalProd + reg.cantidad;
        leer(ventas, reg);
    end;
    writeln(' Total producto: ', totalProd);
    totalSuc := totalSuc + totalProd;
end;
writeln(' Total sucursal: ', totalSuc);
totalEmp := totalEmp + totalSuc;
end;
```

Archivos

Corte de control

```
writeln('Total empresa: ', totalEmp);  
writeln('-----');  
totalGeneral := totalGeneral + totalEmp;  
end;  
  
writeln('TOTAL GENERAL: ', totalGeneral);  
  
close(ventas);  
end.
```

- Merge (unión) de archivos

El **merge** es una técnica utilizada para **combinar información proveniente de varios archivos** que poseen la **misma estructura**.

El objetivo es recorrer simultáneamente todos los archivos y generar un nuevo archivo que integre o resuma la información contenida en ellos.

Este proceso se utiliza, por ejemplo, cuando se tienen **varios archivos detalle** generados por diferentes puestos o sectores y se desea **unificar la información** en un único archivo.

Archivos

Merge

Precondiciones:

- Todos los archivos de entrada tienen la **misma estructura** de registros.
- Todos los archivos están **ordenados por los mismos criterios**.

Para este proceso se utiliza la técnica de **búsqueda del mínimo**, ya vista en la actualización maestro-detalle, pero **sólo para resumir y guardar la información** sin realizar el proceso de actualización de un archivo maestro.

El merge se aplica, por ejemplo, en **algoritmos de clasificación**.

Archivos

Merge 3 archivos

- Ej: generación de archivo de alumnos de un curso

Un instituto dicta un curso que se ofrece en tres sucursales diferentes. Durante el período de inscripción, **cada sucursal registra a los alumnos que se anotan en ese curso y almacena la información en un archivo propio**. El archivo contiene los siguientes datos del alumno:

- Nombre
- DNI
- Dirección
- Carrera

Al finalizar el proceso de inscripción, cada archivo es **ordenado por nombre de alumno** mediante un proceso previo.

Archivos

Merge 3 archivos

El instituto necesita generar un único archivo con todos los alumnos inscriptos en el curso, integrando la información proveniente de las tres sucursales.

Precondiciones:

- El proceso recibe tres archivos con igual estructura.
- Los archivos están ordenados por nombre de alumno.
- Cada alumno aparece una única vez en el conjunto de archivos.

El archivo resultante será un **archivo maestro** del curso que contiene **todos los alumnos inscriptos, ordenado por nombre**.

Archivos

Merge 3 archivos

```
program unionArchivos;  
const  
    valoralto = 'zzzz';  
type  
    str30 = string[30];  
    str10 = string[10];  
    alumno = record  
        nombre: str30;  
        dni: str10;  
        direccion: str30;  
        carrera: str10;  
    end;  
    detalle = file of alumno;  
var  
    det1, det2, det3, maestro: detalle;  
    min, regd1, regd2, regd3: alumno;
```

Archivos

Merge 3 archivos

```
procedure leer (var archivo:detalle; var dato:alumno);  
begin  
    if (not eof( archivo ))  
    then read (archivo, dato)  
    else dato.nombre := valoralto;  
end;
```

```
procedure minimo (var r1,r2,r3:alumno; var min:alumno; var det1,det2,det3: detalle);  
begin  
    if (r1.nombre<r2.nombre) and (r1.nombre<r3.nombre)  
    then begin  
        min := r1;  
        leer(det1,r1)  
    end  
    else if (r2.nombre<r3.nombre)  
    then begin  
        min := r2;  
        leer(det2,r2)  
    end  
    else begin  
        min := r3;  
        leer(det3,r3)  
    end  
end;
```

Archivos

Merge 3 archivos

begin

```
assign (det1, 'det1.dat');    {se asocian y se abren los archivos}
assign (det2, 'det2.dat');
assign (det3, 'det3.dat');
assign (maestro, 'maestro.dat');
reset (det1);
reset (det2);
reset (det3);
rewrite (maestro);

leer(det1, regd1); {se lee el primer elemento y se determina el minimo}
leer(det2, regd2);
leer(det3, regd3);
minimo(regd1, regd2, regd3, min, det1, det2, det3);
```

Archivos

Merge 3 archivos

```
while (min.nombre <> valoralto) do {se procesan todos los archivos}
begin
  write (maestro,min);
  minimo(regd1, regd2, regd3, min, det1, det2, det3);
end;
close (maestro);
close (det1); close (det2); close (det3);
```

```
reset (maestro); {se visualiza en pantalla el nombre y dni de los alumnos inscriptos}
while (not eof(maestro)) do
begin
  read (maestro,min);
  writeln (min.nombre,min.dni);
end;
close (maestro);
```

end.

Archivos

Merge 3 archivos

- Ej: monto acumulado de ventas por vendedor

Un comercio posee **varios archivos con las ventas realizadas durante el día**. Cada archivo corresponde a la caja de un sector distinto del comercio.

Cada elemento del archivo contiene la siguiente información:

- Código de vendedor
- Nombre del producto vendido
- Monto total de la venta

Archivos

Merge 3 archivos

Se desea procesar todos los archivos de ventas y **generar un archivo maestro** que contenga, para cada vendedor, el monto total vendido durante el día.

En el archivo resultante **cada vendedor deberá aparecer una sola vez**, con el total acumulado de sus ventas.

Precondiciones:

- Todos los archivos tienen la **misma estructura**.
- Todos los archivos están **ordenados por código de vendedor**.
- Cada vendedor puede haber realizado varias ventas, por lo que en los archivos detalle pueden existir **varios registros correspondientes al mismo vendedor**.

Archivos

```
program unionArchivos3;  
const  
    valoralto='9999';  
type  
    str4 = string[4];  
    str10 = string[10];  
    vendedor = record  
        codVendedor: str4;  
        producto: str10;  
        montoVenta: real;  
    end;  
    ventas = record  
        codVendedor: str4;  
        total: real;  
    end;  
    maestro = file of ventas;  
    detalle = file of vendedor;
```

Merge 3 archivos

```
var  
    min, reg1, reg2, reg3: vendedor;  
    regm: ventas;  
  
    det1, det2, det3: detalle;  
    mae1: maestro;  
  
    aux: str4;
```

Archivos

Merge 3 archivos

```
procedure leer (var archivo:detalle; var dato:vendedor);
begin
    if (not eof( archivo ))
    then  read (archivo, dato)
    else  dato.codVendedor := valoralto;
end;

procedure minimo (var r1,r2,r3: vendedor; var min:vendedor; var det1,det2,det3:detalle);
begin
    if (r1.codVendedor <= r2.codVendedor) and (r1.codVendedor <= r3.codVendedor)
    then begin
        min := r1;
        leer(det1,r1)
    end
    else if (r2.codVendedor <= r3.codVendedor)
    then begin
        min := r2;
        leer(det2,r2)
    end
    else begin
        min := r3;
        leer(det3,r3)
    end;
end;
```

Archivos

Merge 3 archivos

begin

```
assign (det1, 'ventasCaja1.dat'); {se asocian y se abren los archivos}  
assign (det2, 'ventasCaja2.dat');  
assign (det3, 'ventasCaja3.dat');  
assign (mae1, 'ventas.dat');  
reset (det1);  
reset (det2);  
reset (det3);  
rewrite (mae1);  
  
leer (det1, regd1); {se lee el primer elemento y se determina el minimo}  
leer (det2, regd2);  
leer (det3, regd3);  
minimo(regd1, regd2, regd3, min, det1, det2, det3);
```

Archivos

Merge 3 archivos

```
while (min.codVendedor <> valoralto) do {se procesan los archivos de detalles}
begin
  aux := min.codVendedor;
  regm.codVendedor := min.codVendedor; {valor para registro del archivo maestro}
  regm.total := 0;
  while (min.codVendedor = aux) do {se procesan los registros de un mismo vendedor}
  begin
    regm.total := regm.total + min.montoVenta;
    minimo (regd1, regd2, regd3, min, det1, det2, det3);
  end;
  write(mae1, regm); {se guarda el registro en el archivo maestro}
end;
close(mae1); {se cierran los archivos}
close(det1); close(det2); close(det3);
```

Archivos

Merge 3 archivos

{se visualiza en pantalla el monto acumulado por vendedor}

```
reset (mae1);
```

```
while (not eof( mae1 )) do begin
```

```
    read (mae1,regm);
```

```
    writeln (regm.codVendedor, regm.total);
```

```
end;
```

```
close(mae1);
```

end.

Archivos

Merge N archivos

- Ej: monto acumulado de ventas por vendedor

En el ejercicio anterior se trabajó con un **número fijo** de archivos de ventas, correspondiente a los sectores del comercio.

Sin embargo, en una situación real puede suceder que la **cantidad de sectores no sea conocida de antemano, o que pueda variar con el tiempo.**

En ese caso, el algoritmo debe ser capaz de procesar una cantidad variable de archivos, en lugar de trabajar con un número fijo.

Archivos

Merge N archivos

En **Pascal** los **vectores tienen tamaño fijo**, es decir, no se pueden cambiar en tiempo de ejecución. El tamaño debe conocerse en tiempo de compilación. Para resolver esto, es posible usar **un vector grande junto a una variable lógica de cantidad efectiva**.

En el problema dado, se debe utilizar dos vectores:

- Un **vector de archivos detalle**, que contendrá los archivos de ventas de cada sector.
- Un **vector de registros**, que almacenará el registro actual leído de cada archivo.

El objetivo sigue siendo el mismo que en el ejercicio anterior, pero ahora **el proceso de búsqueda del mínimo deberá realizarse sobre todos los elementos del vector de registros**.

Archivos

```
program unionArchivosN;  
const  
    valoralto='9999';  
    N = 100;  
type  
    str4 = string[4];  
    str10 = string[10];  
    vendedor = record  
        codVendedor: str4;  
        producto: str10;  
        montoVenta: real;  
    end;  
    ventas = record  
        codVendedor: str4;  
        total: real;  
    end;  
    maestro = file of ventas;  
    detalle = file of vendedor;
```

Merge N archivos

```
fileN = array[1..N] of detalle;  
regN = array[1..N] of vendedor;
```

```
var  
    mae1: maestro;  
    deta: fileN;  
  
    regm: ventas;  
    min: vendedor;  
    regDeta: regN;  
  
    aux: str4;  
    i: integer;
```

Archivos

Merge N archivos

```
procedure leer (var archivo:detalle; var dato:vendedor);  
begin  
  
    if (not eof( archivo ))  
    then  
        read (archivo, dato)  
    else  
        dato.cod := valoralto;  
end;
```

Archivos

Merge N archivos

```
procedure minimo (var reg_deta: regN; var min:vendedor;  
                 var deta:fileN);
```

```
var
```

```
    indice_min: integer;
```

```
begin
```

```
    {recorrer el arreglo de registros reg_deta determinando el elemento MINIMO. En la  
    variable indice_min guardar la POSICION del elemento mínimo}
```

```
    determinarMinimo(reg_deta, indice_min);
```

```
    {guardar minimo}
```

```
    min := reg_deta[indice_min];
```

```
    {leer nuevo elemento del archivo correspondiente}
```

```
    leer(deta[indice_min], reg_deta[indice_min]);
```

```
end;
```

Archivos

Merge N archivos

begin

{se preparan los archivos detalle}

for i:= 1 to N

do begin

assign (**deta[i]**, concat('det', IntToStr(i), '.dat'));

reset(**deta[i]**);

leer(**deta[i]**, **regDeta[i]**);

end;

{se prepara el archivo maestro}

assign (mae1, 'maestro.dat');

rewrite (mae1);

{se determina el minimo elemento}

minimo (**regDeta**, min, **deta**);

Archivos

Merge N archivos

```
while (min.codVendedor <> valoralto) do {se procesan los archivos de detalle}
begin
    aux := min.codVendedor;
    {valores para registro del archivo maestro}
    regm.codVendedor := min.codVendedor;
    regm.total := 0;
    {se procesan los reg. de un mismo vendedor}
    while (min.codVendedor = aux) do
    begin
        regm.total := regm.total + min.montoVenta;
        minimo (regDeta, min, deta);
    end;
    {se guarda en el archivo maestro}
    write(mae1, regm);
end;
close (mae1);
```

Archivos

Merge N archivos

{se visualiza en pantalla el monto acumulado por vendedor}

```
reset (mae1);  
while (not eof( mae1 )) do begin  
    read (mae1,regm);  
    writeln (regm. codVendedor, regm.total);  
end;  
close(mae1);
```

end.